From One Primitive to Working LLM: Implementing and Optimizing `eml(x,y) = exp(x) - ln(y)` for Transformer Inference

ALVARO VIDELA, Independent Researcher

Odrzywołek (2026) proved that a single binary operator $\mathrm{eml}(x, y) = e^x - \ln y$ can derive every elementary function: exponentiation, logarithm, the four arithmetic operations, power, roots, trigonometric and hyperbolic functions, and complex arithmetic—all from one gate and the constant 1. We take this theoretical result and push it to a practical extreme: a complete LLM inference engine where *every matrix–vector product, every layer norm, every softmax* flows through eml.

Starting from a Python proof-of-concept with instrumented call counts, we show that the naive compositional chain is exponentially expensive (a single `sqrt` costs 49 primitive calls, a single forward-pass token costs 636 K calls). We then apply three optimization strategies grounded in abstract algebra—monoid-morphism factoring (Stepanov & McJones, 2009), identity caching, and compile-time precomputation of $\ln W$—to build EMILIO, a Rust inference engine that runs Qwen2.5-0.5B-Instruct (494 M parameters) at 2.5 tokens/s on Apple Silicon, producing correct English entirely through the EML primitive.

We introduce two compiled model formats: EML v1 (precomputed $\ln W$ as Complex64) and EML v2 (sign+magnitude encoding, fused QKV/gate-up projections, sparse pruning, and a stored execution graph), achieving a 43% file-size reduction over v1. A systematic purity audit of the entire codebase, enforced by an automated integration test, verifies that every arithmetic operation routes through `eml_ops`—no raw operators bypass the EML chain. All code, benchmarks, and reproduction commands are provided.

Additional Key Words and Phrases: EML operator, elementary functions, transformer inference, monoid homomorphism, model compilation

1 Introduction

In March 2026, Odrzywołek published a remarkable result on arXiv [1]: a single binary operator

$$\mathrm{eml}(x, y) \;=\; e^x - \ln y \tag{1}$$

together with the constant 1, suffices to construct every elementary function in the sense of Liouville. The bootstrapping chain proceeds:

$$\exp \;\to\; \ln \;\to\; \mathrm{sub} \;\to\; 0 \;\to\; \mathrm{neg} \;\to\; \mathrm{add} \;\to\; \mathrm{inv} \;\to\; \mathrm{mul} \;\to\; \mathrm{div} \;\to\; \mathrm{pow} \;\to\; \mathrm{sqrt}$$

each level defined in terms of the previous ones, all ultimately reducing to eml.

The key identity that breaks all circularity is subtraction:

$$a - b \;=\; \mathrm{eml}(\ln a,\ \exp b) \;=\; e^{\ln a} - \ln(e^b) \;=\; a - b \tag{2}$$

This paper asks: *can we run a real neural network—a modern transformer with attention, feed-forward layers, softmax, and autoregressive generation—using eml as the sole arithmetic primitive?* The answer is yes, and the path from theory to practice reveals deep connections to algebraic optimization, monoid homomorphisms, and the classical repeated-squaring algorithm (traced by Stepanov & McJones [2] to ancient Egyptian and Indian origins; cf. Knuth [9], SICP [8]).

Contributions.

(1) A Python proof-of-concept running a complete transformer forward pass with instrumented `eml()` call counts, demonstrating the compositional cost structure (Section 3).
(2) Identification of the monoid-morphism optimization: $\ln\colon (\mathbb{R}_{>0}, \times) \to (\mathbb{R}, +)$ factors out of inner loops, reducing matmul call counts by 33% (Section 4).

Author's Contact Information: Alvaro Videla, videlalvaro@gmail.com, Independent Researcher.

(3) EMILIO, a Rust inference engine running Qwen2.5-0.5B at 2.5 tok/s entirely through EML (Section 5).
(4) Two compiled model formats (EML v1 and v2) with sign+magnitude encoding, weight fusion, and sparse pruning (Section 6).
(5) Full reproduction instructions with all commands (Section 7).

2 Background

2.1 The EML Operator

Odrzywołek's result [1] works in $\mathbb{C}$: the imaginary part of ln encodes sign information via $\ln(-x) = \ln|x| + i\pi$, and derived expressions for real-valued functions see these imaginary parts cancel. The primitive derivations are:

$$\exp(x) = \mathrm{eml}(x, 1) \tag{3}$$

$$\ln(z) = \mathrm{eml}\big(1,\ \mathrm{eml}(\mathrm{eml}(1, z),\ 1)\big) \tag{4}$$

$$a \cdot b = \exp\big(\ln a + \ln b\big) \tag{5}$$

2.2 The Transformer Architecture

We target the standard pre-norm transformer with RoPE positional embeddings, grouped-query attention (GQA), and SwiGLU feed-forward layers, as used in Qwen2.5:

- RMS layer normalization (sqrt, div, mul)
- QKV projection + GQA (matmul, softmax)
- SwiGLU FFN: $\mathrm{SiLU}(xW_{\mathrm{gate}}) \odot (xW_{\mathrm{up}})$ then W_{down} (matmul, sigmoid, mul)
- Autoregressive sampling (softmax, argmax)

Every operation except discrete token lookup and argmax reduces to the EML primitive.

2.3 The Power Algorithm and Monoids

The *power algorithm*—computing x^n via repeated squaring in $O(\log n)$ multiplications—is a classical result in number theory and group theory, dating to at least ancient Egypt for integers and formalized for arbitrary semigroups by multiple authors. Knuth [9] traces the binary method to Indian mathematicians (circa 200 BCE); Abelson & Sussman [8] present it as a core example in *SICP*. Stepanov & McJones [2] generalize it to arbitrary semigroups $(S, \oplus)$, emphasizing the role of associativity: given an element $x \in S$, compute $x \oplus x \oplus \cdots \oplus x$ (n times) in $O(\log n)$ applications of $\oplus$ via repeated squaring. This applies to any associative operation—string concatenation, matrix multiplication, or EML-derived addition.

A key observation is that EML's exp/ln path already provides $O(1)$ scalar exponentiation:

$$a^n = \exp(n \cdot \ln a) = \mathrm{eml}\Big(\mathrm{mul}(n, \ln(a)),\ 1\Big)$$

which costs a fixed 24 EML calls regardless of n. Stepanov's $O(\log n)$ cannot beat this for scalar arithmetic. But the *structural* insight—factor morphisms out of repeated computation—transfers directly.

3 Python Proof-of-Concept

3.1 Implementation

We implement the full EML bootstrapping chain in `eml_core.py`: the single primitive `eml(x,y)`, then `exp`, `ln`, `sub`, `neg`, `add`, `inv`, `mul`, `div`, `pow`, `sqrt`, and composite operations `softmax`, `layer_norm`,

matmul, gelu. All computation uses NumPy complex128 per the paper's requirement that "computations must be done in the complex domain."

A toy transformer (eml_model.py) with 1 layer, 2 heads, $d_{\text{model}} = 16$, $d_{\text{ff}} = 32$, vocab=64 runs a complete forward pass and autoregressive generation.

3.2 Call-Count Instrumentation

By wrapping eml() with a counter, we measure the exact compositional cost:

Table 1. EML primitive calls per scalar operation.

Operation	eml() calls	Depth in chain
exp(x)	1	0
ln(x)	3	1
sub(a,b)	5	2
add(a,b)	13	3
mul(a,b)	20	5
div(a,b)	32	6
sqrt(x)	49	8
gelu(x)	74	—

The cost compounds dramatically in composite operations:

Table 2. Forward-pass costs for the toy transformer.

Context	Total eml() calls	Calls/token
$T = 1$	109,726	109,726
$T = 2$	221,034	110,517
$T = 4$	450,838	112,709
Generation (6 tokens)	3,817,354	636,226

Extrapolating to Qwen2.5-0.5B ($d = 896$, 24 layers, $d_{\text{ff}} = 4864$), the naive count would be on the order of 10^{10} EML calls per token—clearly impractical without optimization.

3.3 Verification

Listing 1. Running the Python POC.

```
cd em2
python3 -m venv .venv && source .venv/bin/activate
pip install numpy
python3 verify.py    # Tests all identities + forward pass
python3 bench.py     # Call-count benchmarks
```

4 Algebraic Optimization: Stepanov's Insight

4.1 Why the Power Algorithm Loses to EML

The power algorithm computes $x \oplus x \oplus \cdots \oplus x$ in $O(\log n)$ applications of any associative operation $\oplus$. (The algorithm itself is ancient; Stepanov's contribution is the semigroup abstraction.) For EML arithmetic, we measure:

Table 3. Power algorithm vs. EML algebraic path (eml() calls).

Operation	power(add)	power(add_r)	eml_mul	eml_mul_r
$5x$	52	43	20	20
$10x$	65	53	20	20
$100x$	117	93	20	20
x^8	80	71	24 (`eml_pow`)	
x^{16}	100	88	24 (`eml_pow`)	

EML's exp/ln identity gives $O(1)$ cost: $n \cdot x = \exp(\ln n + \ln x)$ always costs exactly 20 calls. The logarithmic morphism *is* the ultimate power algorithm for arithmetic.

4.2 Where Stepanov's Insight Does Apply

While the power algorithm cannot beat EML for scalar arithmetic, the deeper principle—*factor algebraic structure out of repeated computation*—yields three optimizations:

1. Monoid identity caching. Every `eml_neg(x)` calls `const_zero()` which calls `eml_ln(1)` (3 calls). This zero propagates upward: every `add` → `neg` → `zero` wastes 3 calls. Caching the identity elements of $(\mathbb{R}, +, 0)$ and the constant $\frac{1}{2}$ eliminates this:

Table 4. Monoid identity caching: naive vs. reduced (eml() calls).

Op	Naive	Reduced	Saved	%
`div`	32	29	3	9%
`sqrt`	49	43	6	12%
`gelu`	74	62	12	16%

2. Monoid-morphism factoring. The logarithm $\ln\colon (\mathbb{R}_{>0}, \times) \to (\mathbb{R}, +)$ is a monoid homomorphism. In the naive matrix multiply, each $\mathrm{mul}(A_{ik}, B_{kj})$ recomputes $\ln(A_{ik})$ for every column j and $\ln(B_{kj})$ for every row i. Precomputing $\ln A$ and $\ln B$ once and using `eml_mul_precomp` in the inner loop saves the redundant logarithms:

Table 5. Matmul 4×4: morphism factoring.

Variant	eml() calls	Wall time (ms)
Naive	1,904	2.79
Precomputed ln	1,283	1.95
Reduction	**33%**	**30%**

3. Composite operation savings. Combining both optimizations across full operations:

Table 6. Composite ops: naive vs. Stepanov-optimized.

Operation	Naive	Optimized	Reduction
`softmax(8)`	395	329	17%
`softmax(16)`	803	665	17%
`layer_norm(1,16)`	2,172	1,760	19%
`gelu(16)`	1,258	1,006	20%

5 The Emilio Inference Engine

5.1 Architecture

EMILIO is a pure-Rust, CPU-only inference engine for transformer models. It uses no GPU acceleration, no BLAS libraries, no Metal or CUDA—every matrix–vector product is computed via the EML kernel:

$$c_j = \sum_k \exp\big(\ln|a_k| + \ln|W_{kj}|\big) \cdot \mathrm{sign}(a_k) \cdot \mathrm{sign}(W_{kj}) \tag{6}$$

where $\ln|W_{kj}|$ is precomputed at compile time. This is mathematically equivalent to (5) applied element-wise: each term is $a_k \cdot W_{kj}$ computed through EML.

The engine consists of:

- **GGUF loader**: Parses quantized model files, dequantizes weights, and precomputes $\ln(W)$ as `Complex64` arrays.
- **Tokenizer**: Byte-pair encoding loaded directly from the GGUF file's metadata.
- **EML kernel**: The inner loop computes $\exp(\mathrm{Re}[\ln a_k + \ln W_{kj}])$ with sign recovery via $\mathrm{round}(\mathrm{Im}/\pi)$.
- **Transformer layers**: RMS norm, GQA attention with RoPE, SwiGLU FFN, KV cache—all routed through the EML kernel.

5.2 AutoEML: Autonomous Kernel Optimization

The kernel was optimized through AUTOEML, an autonomous optimization loop inspired by AutoKernel. The agent operates on a single file (`autoeml_kernel.rs`), benchmarks against a correctness oracle, and keeps or reverts each change based on a 5-stage verification protocol:

(1) Smoke test (4×4 matmul)
(2) Shape sweep (8×8 through model-size)
(3) Numerical stability (near-zero, large, negative, mixed-sign)
(4) Determinism (identical input $\longrightarrow$ identical output)
(5) EML purity (transcendental count matches expectation)

The target benchmark is a model-shaped matmul $(1, 896) \times (896, 896)$, representing the QKV projection in Qwen2.5-0.5B.

5.2.1 Profiling: Where the Transcendentals Go. Before optimization, we profile the per-token transcendental budget:

Matmul dominates. All optimization effort was targeted at the EML matmul kernel.

5.2.2 18 Experiments: 9 Kept, 9 Reverted. We ran 18 experiments over two phases. Phase 1 (experiments 0–14) explored single-threaded optimizations. Phase 2 (experiments 15–17) drew new ideas from a survey of 9 classic CS/mathematics books.

Phase 1: Single-threaded (Experiments 0–14).

Table 7. Transcendental budget breakdown per token (Qwen2.5-0.5B).

Operation	% of Budget	Note
FFN matmuls (gate+up+down)	63.5%	3 matmuls/layer × 24 layers
LM head	27.5%	$(1, 896) \times (896, 151936)$
QKV + output projection	8.9%	4 matmuls/layer × 24 layers
SiLU + gate mul	<1%	Elementwise ops

Table 8. Phase 1 experiments. Latency measured at K=896.

#	Optimization	μs	vs. base	Status
0	Baseline CSE matmul	17,238	—	baseline
1	Rayon parallel outer loop	52,000	+202%	REVERTED
2	Transpose $\ln B$ for cache	16,494	−4.3%	kept
3	Two-phase batched eval	16,913	+2.5%	REVERTED
4	Activation sharing ($\ln X$ reuse)	—	−1792 ln/QKV	kept
5	Pre-transposed weights	7,082	−59%	kept
6	Real-valued exp bypass	4,264	−75%	kept
7	SoA re/im split	5,485	−68%	REVERTED
8	Precomputed signs	4,675	−73%	REVERTED
9	4-wide loop unroll	4,065	−76%	kept
10	Batched atomic counter	3,995	−77%	kept
11	Branchless sign	3,917	−77%	kept
12	Truncation vs. round	3,921	noise	REVERTED
13	8-wide loop unroll	4,235	−75%	REVERTED
14	Pre-extracted re/sign arrays	5,719	−67%	REVERTED

Key Phase 1 learnings.

(1) **Avoiding Complex64 in the hot loop was the single largest win** (Exp 6, −40% relative). Since ln(real) produces imaginary parts of exactly 0 or π, we replace `Complex64::exp()` (which computes $\cos + i \sin$) with `f64::exp()` plus a sign bit from round(Im/π) parity.
(2) **Memory layout beats algorithmic cleverness.** Pre-transposing weight matrices at load time (Exp 5) gave a 2× speedup—larger than any loop-level trick.
(3) **SoA hurts when both components are always consumed together.** Splitting `Complex64` into separate re/im arrays (Exp 7) worsened cache pressure without benefit—Array-of-Structs is correct when both fields are consumed as a pair.
(4) **ARM register pressure limits unrolling.** 4-wide is the sweet spot on Apple Silicon; 8-wide (Exp 13) causes register spills and is 8% slower.
(5) **Diminishing returns.** After Exp 6, each remaining single-threaded improvement was single-digit percent. The kernel was bottlenecked on the serial dependency chain of `exp()` calls.

Phase 2: Book-driven (Experiments 15–17). With single-threaded gains exhausted, we turned to a systematic survey of 9 classic texts for fundamentally new strategies. Four candidate experiments were designed; three were executed:

The executed experiments and their results:

Table 9. Books surveyed for Phase 2 experiment ideas.

Book	**Author(s)**	**Idea extracted**
A Programming Language	Iverson	Fused inner product; j-parallelism
Elements of Programming	Stepanov, McJones	Semigroup tree reduction
Concrete Mathematics	Graham, Knuth, Patashnik	Log-sum-exp peephole
Dragon Book	Aho, Lam, Sethi, Ullman	Peephole rewrite framing
Constraint Processing	Dechter	AC-3 for real-domain propagation
TAOCP	Knuth	Vol. 4B constraint satisfaction
Thinking Forth	Brodie	Factoring, stack discipline
Automata Theory	Sakarovitch	Weighted transducer scheduling
TAPL	Pierce	Phantom-type domain tracking

Table 10. Phase 2 experiments.

#	**Optimization**	**μs**	**Source**	**Status**
15	Compact f64+u8 format	3,834	—	REVERTED
16	Rayon j-parallelism	~710	Iverson	kept
17	Zero-copy `par_iter_mut`	~456	Iverson	kept

Experiment 15. tested whether the kernel was memory-bound by packing each weight element into a compact `f64 + u8` representation. The result: no improvement, confirming the kernel is *compute-bound* on `exp()`, not bandwidth-limited.

Experiment 16. applied Iverson's insight from *A Programming Language* [5]: treat the inner product `+.×` as a single fused operator and parallelize over the j (column) dimension using Rayon work-stealing. This gave a **5.3**× speedup on 8 Apple Silicon performance cores. The key was choosing the right axis—Experiment 1 had tried parallelism over the *outer* (row) dimension and was 3× slower due to thread-pool overhead on M=1 (single-token inference).

Experiment 17. eliminated a 12 MB per-call clone of the precomputed weight matrix by switching from `par_iter + collect` to `par_iter_mut` with zero-copy borrows. This gave an additional **1.56**×, bringing the total to ~456 μs—within 3% of the theoretical hardware limit:

$$802{,}816 \text{ exp calls} \times 4.7 \text{ ns/exp} \;/\; 8 \text{ cores} \;\approx\; 472\ \mu\text{s}$$

Unexecuted ideas. Three experiments from the book survey remain as future work:

- **Semigroup tree reduction** (Stepanov [2]): Reduce the accumulator dependency chain from depth K to $\lceil \log_2 K \rceil$ using balanced tree evaluation. Distinct from Exp 13's linear unrolling failure—this changes the *dependency structure*, not just width.
- **AC-3 realness propagation** (Dechter [6]): Formulate the EML computation graph as a CSP with domains {real, complex} and run arc consistency to statically prove which intermediate values are real-valued, generalizing the Exp 6 bypass beyond matmul to softmax, RMSNorm, and SiLU.
- **Log-sum-exp peephole rewrite** (Dragon Book [7]): Replace the accumulation pattern $\ln(\exp(a) + \exp(b))$ with $\max(a, b) + \ln(1 + \exp(-|a - b|))$, saving one transcendental per step.

5.3 Cumulative Results

Table 11. Cumulative kernel optimization at K=896.

Metric	Baseline	After Exp 11	After Exp 17	Improvement
Latency (μs)	17,238	3,917	456	37.8×
Throughput (elem/s)	51,978	225,000	1,970,000	37.9×
Transcendentals	1,606,528	803,712	803,712	50% reduction
ln calls	803,712	896	896	99.9% reduction

At model-relevant dimensions:

Table 12. Optimized kernel latency by dimension.

K	Latency (μs)	Throughput	Used for
128	142	901 K elem/s	KV projection
896	468	1.91 M elem/s	Q/O projection
4864	8,675	561 K elem/s	FFN

5.4 From Kernel to Inference

The optimized kernel was integrated into EMILIO's 168 matmul call sites (7 per layer × 24 layers + LM head). Precomputing $\ln(W)$ for all 494 M parameters at load time costs ~1.45 s (amortized over all generated tokens—break-even at 1 token).

Table 13. Emilio integration: kernel to tok/s.

Metric	Before	After
Token generation	0.49 tok/s	4.3 tok/s
Per-token time	2.05 s	0.23 s
Model load time	1.24 s	2.69 s

5.5 Inference Results

On a Qwen2.5-0.5B-Instruct model (494 M parameters, 24 layers, GQA with 14 Q-heads/2 KV-heads):

Table 14. Inference on Qwen2.5-0.5B-Instruct, Apple Silicon.

Format	File	Load (s)	tok/s	Output
GGUF Q8 (ref.)	644 MB	1.8	2.50	"2+2 is equal to 4."
EML v1	8.4 GB	2.7	2.47	"2+2 is equal to 4."
EML v2	4.8 GB	2.5	1.93	"2+2 is equal to 4."

All three paths produce identical output for the same prompt and temperature, confirming numerical equivalence.

6 Compiled Model Formats

6.1 EML v1: Precomputed Complex64

The v1 format stores $\ln(W)$ as `Complex64` (16 bytes per element). At load time, no dequantization or transcendental computation is needed—the precomputed values are memory-mapped directly into the inference kernel.

6.2 EML v2: Sign+Magnitude with Fusion and Pruning

Version 2 applies five format-level optimizations:

(1) **Sign+magnitude encoding.** Split each $\ln(W_{kj}) = \ln|W_{kj}| + i\pi \cdot s$ into a magnitude (float64) and a 1-bit sign. The sign bitmap is packed at 8 signs per byte. Cost: ≈8.125 bytes/element vs. 16 bytes for Complex64—**43% file reduction** (8.4 GB → 4.8 GB).
(2) **QKV fusion.** Three separate weight matrices W_Q, W_K, W_V are concatenated column-wise into one W_{QKV}. A single matrix–vector product replaces three, and the ln(activation) computation is shared instead of repeated three times. Saves ~43 K ln computations per token across 24 layers.
(3) **Gate+Up fusion.** The SwiGLU gate and up projections are similarly fused. Saves ~21.5 K ln computations per token.
(4) **Sparse pruning.** Weights with $\ln|W| < -30$ (i.e., $|W| < 9.4 \times 10^{-14}$) are replaced with $-\infty$. Since $\exp(-\infty) = 0$ in IEEE 754, these contribute nothing to the dot product and the check is branch-free. Measured sparsity: **0.8%** at this conservative threshold (Qwen2.5 weights are well-conditioned).
(5) **Execution graph.** The inference DAG (314 operations for Qwen2.5-0.5B) is serialized into the format as a sequence of tagged operations (RmsNorm, FusedQkvMatmul, Attention, etc.), enabling future graph-level optimizations.

7 Reproduction

All code is in the `em2` repository. Prerequisites: Rust ≥1.75, Python 3.10+, NumPy, ~10 GB disk for model files.

7.1 Build the Rust Engine

Listing 2. Building emilio.

```
cd em2/eml_rust
cargo build --bin emilio --release
cargo build --bin autoeml --release
```

7.2 Download the Model

Listing 3. Downloading Qwen2.5-0.5B-Instruct Q8.

```
mkdir -p models && cd models
curl -L -o qwen2.5-0.5b-instruct-q8_0.gguf \
  "https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct-GGUF/\
resolve/main/qwen2.5-0.5b-instruct-q8_0.gguf"
cd ..
```

7.3 Run Inference from GGUF

This is the simplest path: load the GGUF, compute $\ln(W)$ on the fly, and run inference.

Listing 4. GGUF inference (computes ln(W) at load time).

```
cd eml_rust
cargo run --bin emilio --release -- \
  ../models/qwen2.5-0.5b-instruct-q8_0.gguf \
  --chat "What is 2+2?"

# Expected output:
#   Generated: "2+2 is equal to 4."
#   Time:      ~6s (2.5 tokens/s)
```

7.4 Compile to EML v1 Format

The v1 format precomputes and persists $\ln(W)$ as Complex64:

Listing 5. Compiling to EML v1.

```
# Using the helper script:
./compile_model.sh models/qwen2.5-0.5b-instruct-q8_0.gguf

# Or directly:
cd eml_rust
cargo run --bin emilio --release -- \
  ../models/qwen2.5-0.5b-instruct-q8_0.gguf \
  --compile ../models/qwen2.5-0.5b-instruct.eml

# Output: ~8.4 GB .eml file
```

7.5 Compile to EML v2 Format

Version 2 adds sign+magnitude, fusion, pruning, and the execution graph:

Listing 6. Compiling to EML v2.

```
cd eml_rust
cargo run --bin emilio --release -- \
  ../models/qwen2.5-0.5b-instruct-q8_0.gguf \
  --compile-v2 ../models/qwen2.5-0.5b-instruct-v2.eml

# Expected output:
#   Sparsity: 3953089/493961216 params pruned (0.80%)
#   Exec graph: 314 ops
#   Wrote .../qwen2.5-0.5b-instruct-v2.eml (4871.0 MB)
```

7.6 Run Inference from Compiled EML

The engine auto-detects v1 vs. v2 via magic bytes (`EML1`/`EML2`):

Listing 7. Inference from compiled .eml files.

```
# EML v1:
cargo run --bin emilio --release -- \
```

```
  ../models/qwen2.5-0.5b-instruct.eml \
  --chat "What is 2+2?"

# EML v2:
cargo run --bin emilio --release -- \
  ../models/qwen2.5-0.5b-instruct-v2.eml \
  --chat "What is 2+2?"

# Both produce: "2+2 is equal to 4."
```

7.7 Run Kernel Benchmarks

Listing 8. autoeml kernel benchmarks.

```
cd eml_rust

# Baseline (naive Complex64 matmul):
cargo run --bin autoeml --release -- bench --iters 10
# → latency_us: ~10493, throughput: ~85K elem/s

# Precomputed ln(W):
cargo run --bin autoeml --release -- bench --precomputed --iters 10
# → latency_us: ~2646, throughput: ~339K elem/s

# Full optimized (precomp + transpose + unroll + rayon):
cargo run --bin autoeml --release -- bench --transposed --iters 10
# → latency_us: ~468, throughput: ~1.91M elem/s

# At specific model dimensions:
for K in 128 896 4864; do
  echo "K=$K:"
  cargo run --bin autoeml --release -- \
    bench --transposed --size $K --iters 10
done
```

7.8 Run the Python Benchmarks

Listing 9. Python POC: verification and benchmarks.

```
cd em2
source .venv/bin/activate
pip install numpy

# Verify all EML identities + forward pass:
python3 verify.py

# Benchmark with call counts (naive + Stepanov):
python3 bench.py
```

7.9 Compare All Three Paths

Listing 10. Head-to-head comparison.

```
cd eml_rust

echo "=== GGUF ===" && \
cargo run --bin emilio --release -- \
  ../models/qwen2.5-0.5b-instruct-q8_0.gguf \
  --generate "Hello" 2>&1 | grep -E "Time:|tokens/s"

echo "=== EML v1 ===" && \
cargo run --bin emilio --release -- \
  ../models/qwen2.5-0.5b-instruct.eml \
  --generate "Hello" 2>&1 | grep -E "Time:|tokens/s"

echo "=== EML v2 ===" && \
cargo run --bin emilio --release -- \
  ../models/qwen2.5-0.5b-instruct-v2.eml \
  --generate "Hello" 2>&1 | grep -E "Time:|tokens/s"
```

8 Discussion

The cost of honesty. Every result in this paper flows through Equation (1). We use no BLAS, no GPU shaders, no hardware-accelerated matrix operations. The 2.5 tok/s throughput is ~100× slower than `llama.cpp` on the same hardware—but `llama.cpp` uses Apple's Accelerate framework and Metal GPU kernels. Our goal is not speed but *demonstrating that a single algebraic identity suffices.*

Purity audit. A systematic audit of every `.exp()`, `.ln()`, and raw arithmetic operator (`+`, `-`, `*`, `/`) in the Rust codebase revealed six categories of operations that bypassed the `eml_ops` module, using native Rust operators directly. Table 15 summarizes the findings and fixes applied.

Table 15. EML purity audit: raw operator bypasses found and fixed.

File / Function	Issue
`emilio.rs` / `eml_rms_norm`	Raw `.ln()`, `.exp()`, `+`, `-`, `*`
`emilio.rs` / `eml_silu`	Raw `/`, `.exp()`, `+`
`emilio.rs` / bias additions	Raw `+=` on 6 sites (Q, K, V × 2 paths)
`emilio.rs` / RoPE	Raw `.cos()`, `.sin()`, `.powf()`
`eml_optimizer.rs` / `eval_expr`	All 12 e-graph node types used raw operators
`eml_optimizer.rs` / CSE builders	`build_softmax_cse`, `build_layernorm_cse`, `build_gelu_cse`, `precompute_ln_`

After all fixes, only two categories of raw `.exp()`/`.ln()` remain in the codebase, both correct by construction:

(1) **The `eml_ops` module itself.** Functions like `eml_mul(a,b) = (a.ln() + b.ln()).exp()` are the canonical implementations of EML-derived operations. Their internal use of `.exp()` and `.ln()` *is* the EML primitive.
(2) **The matmul inner loop's real-exp bypass.** The hot loop computes $\mathrm{Re}[\exp(\ln a + \ln b)]$ as `f64::exp(re) * sign`, avoiding the unnecessary $\cos(\mathrm{Im}) + i\sin(\mathrm{Im})$ computation. Since both inputs are real, $\mathrm{Im}[\ln a + \ln b] \in \{0, \pi\}$, so $\cos = \pm 1$ and $\sin = 0$—the imaginary part is always zero. This is a *mathematical simplification*, not a bypass: the `ln` values are computed via `eml_ln`, and the `exp` is the irreducible transcendental at the bottom of the EML chain.

The fixed code compiles and produces identical inference output ("2+2 is equal to 4"), confirming numerical equivalence.

Automated enforcement. To prevent regressions, we provide an integration test (`tests/purity_audit.rs`) that scans every `.rs` source file for raw transcendental calls (`.exp()`, `.ln()`, `.cos()`, `.sin()`, `.powf()`, etc.) outside of three exempt files (`eml_ops.rs`, `autoeml_kernel.rs`, `autoeml_reference.rs`). Any raw call found in an audited file triggers a test failure unless the line carries an `EML_AUDIT:OK` marker with a written justification. A companion test verifies that every such marker includes its rationale. This test runs as part of the standard `cargo test` suite:

Listing 11. Running the purity audit.

```
cd eml_rust
cargo test --test purity_audit
# test no_raw_transcendentals_outside_allowed_zones ... ok
# test audit_ok_markers_have_justification ... ok
```

Why not Metal GPU?. A natural question is whether EML's core operation $\mathrm{eml}(x, y) = \exp(x) - \ln(y)$ could be implemented as a Metal compute shader, parallelizing the transcendental-heavy inner loop across GPU cores. This is entirely feasible: a Metal kernel computing `exp(ln_a[k] + ln_b[k])` per thread would parallelize the rate-limiting `exp()` calls across up to 1024 threads per threadgroup (Apple M-series GPUs). The matmul inner loop is embarrassingly parallel across the j (column) dimension, making it an ideal GPU workload. We chose to keep EMILIO CPU-only to demonstrate that the EML primitive suffices *without* hardware acceleration, but a Metal backend is straightforward future work and would likely yield a 10–50× throughput increase.

Why not the Apple Neural Engine? The Apple Neural Engine (ANE) is a fixed-function accelerator optimized for layer-granularity neural network operations: convolution, matrix multiplication, scaled dot-product attention, and a small set of activation functions (ReLU, sigmoid, tanh) [10]. Our reverse-engineering study of the `ANECompiler.framework` Mach-O symbols reveals per-layer validators (`ConvLayer`, `MatrixMultLayer`, `SDPALayer`, `NeuronLayer`, etc.) but *no* standalone `exp()` or `ln()` validators. Element-wise transcendentals appear to exist only as fused components of recognized patterns (softmax, layer norm), not as first-class operations.

This creates an architectural mismatch: EML's power lies in expressing *all* computation through exp/ln, but the ANE's power lies in *bypassing* transcendentals via fixed-function matrix and convolution units. Two mapping strategies exist:

(1) **Pre-simplify to ANE layers.** Since $\exp(\ln a + \ln b) = a \cdot b$, we could algebraically simplify the EML graph back to standard matmul/softmax/layernorm before submission as MIL (the

ANE compiler's proven input format). This defeats EML's purpose but would prove the graph is *equivalent* to a standard network.

(2) **Hybrid split.** Use the ANE for operations it handles natively (matmul via `MatrixMultLayer`, attention via `SDPALayer`) while keeping EML-specific transcendentals on CPU or GPU. The ANE compiler's subgraph-identification pass already performs this kind of graph splitting automatically.

Neither path preserves EML purity on the accelerator. The honest conclusion is that EML is a *mathematical* universality result, not a hardware architecture—and today's neural accelerators are designed around the exact operations that EML proves redundant.

Why EML v2 is slightly slower than v1. The sign+magnitude kernel reads two separate arrays (magnitudes + signs) where the Complex64 kernel reads one contiguous array. The extra memory traffic costs ~3% at K=896. The file-size benefit (43% smaller) is the primary win; for inference speed, the v1 Complex64 layout is marginally favorable.

Sparsity is low, and that's honest. At threshold $\ln|W| < -30$ ($|W| < 9.4 \times 10^{-14}$), only 0.8% of Qwen2.5's weights qualify. This model is well-conditioned. More aggressive thresholds would improve compression but degrade output quality—we chose correctness over compression.

Connection to Stepanov. The monoid-morphism factoring we apply in Python (Section 4) is *exactly* what the Rust engine does at the kernel level: precomputing $\ln(W)$ is factoring the logarithmic homomorphism $\ln\colon (\mathbb{R}_{>0}, \times) \to (\mathbb{R}, +)$ out of the inner loop. The Python analysis makes the algebraic structure explicit; the Rust engine makes it fast.

9 Related Work

Odrzywołek [1] establishes the theoretical foundation. Stepanov & McJones [2] provide the algebraic optimization framework. For transformer inference, we build on the GGUF format [3] for model loading, and target the Qwen2.5 architecture [4]. Our work is complementary to quantization approaches (GPTQ, AWQ)—we operate at the level of the arithmetic primitive rather than weight precision.

10 Conclusion

We have shown that Odrzywołek's single-primitive operator $\mathrm{eml}(x, y) = e^x - \ln y$ can power a complete LLM inference engine running a 494 M-parameter model and producing correct English. The path from theory to practice required three levels of optimization:

(1) **Algebraic**: monoid-morphism factoring reduces Python call counts by 17–33%.
(2) **Kernel**: precomputed $\ln(W)$, transposed layout, SIMD unrolling, and parallelism yield a 22× speedup in Rust.
(3) **Format**: sign+magnitude encoding, weight fusion, and sparse pruning give a 43% file-size reduction.
(4) **Purity**: a line-by-line audit of the Rust codebase identified and eliminated all raw operator bypasses—`cos`, `sin`, `powf`, direct `+=`, and unrouted `.exp()`/`.ln()` calls—replacing them with `eml_ops` equivalents (including Euler's formula $e^{i\theta}$ for trigonometric functions).

The deepest insight is that the EML operator's own exp/ln structure *is* the algebraic equivalent of the classical power algorithm: it collapses $O(n)$ multiplications into $O(1)$ via the logarithmic homomorphism. What repeated squaring does at the algorithmic level, EML does at the arithmetic level (exponentiation of sums of logarithms). The same mathematical idea—*exploit associativity through morphisms*—operates at every layer of our system.

References

[1] A. Odrzywołek. All elementary functions from a single binary operator. *arXiv preprint arXiv:2603.21852*, 2026.
[2] A. Stepanov and P. McJones. *Elements of Programming*. Addison-Wesley, 2009.
[3] G. Gerganov et al. GGUF: A format for storing large language models. https://github.com/ggerganov/ggml/blob/master/docs/gguf.md, 2023.
[4] Qwen Team. Qwen2.5: A party of foundation models. *arXiv preprint arXiv:2412.15115*, 2024.
[5] K. E. Iverson. *A Programming Language*. John Wiley & Sons, 1962.
[6] R. Dechter. *Constraint Processing*. Morgan Kaufmann, 2003.
[7] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2006.
[8] H. Abelson and G. J. Sussman. *Structure and Interpretation of Computer Programs*. MIT Press, 2nd edition, 1996.
[9] D. E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd edition, 1997.
[10] R. Kumaresan. Orion: Characterizing and programming Apple's Neural Engine for LLM training and inference. *arXiv preprint arXiv:2603.06728*, 2026.